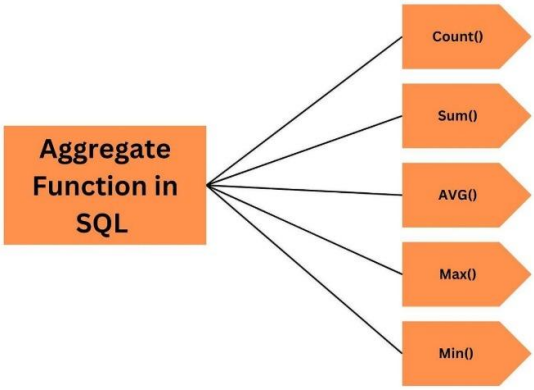
STYLECART -FASHION E-COMMERCE DATABASE MANAGEMENT SYSTEM

SALES AND CUSTOMER ANALYTICS SYSTEM

1.Aggregate functions

Aggregate functions in SQL are used to perform calculations on multiple rows and return a single value as the result.



1.1COUNT()

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS
  2  FROM Orders;

TOTAL_ORDERS
------------
           6
```

1.2SUM()

```
SQL> SELECT SUM(Total_Amount) AS TOTAL_SALES
  2  FROM Orders;

TOTAL_SALES
-----------
      12500
```

1.3AVG()

```
SQL> SELECT ROUND(AVG(Total_Amount), 2) AS AVERAGE_SALES
  2  FROM Orders;

AVERAGE_SALES
-------------
      2083.33
```

1.4MIN()

```
SQL> SELECT MIN(Total_Amount) AS MINIMUM_SALES
  2  FROM Orders;

MINIMUM_SALES
-------------
         1500
```

1.5MAX()

```
SQL> SELECT MAX(Total_Amount) AS MAXIMUM_SALES
  2  FROM Orders;

MAXIMUM_SALES
-------------
         3000
```

2.Generate Total Sales Reports

```
SQL> SELECT
  2      COUNT(Order_ID) AS TOTAL_ORDERS,
  3      SUM(Total_Amount) AS TOTAL_SALES,
  4      ROUND(AVG(Total_Amount), 2) AS AVERAGE_SALES,
  5      MIN(Total_Amount) AS MINIMUM_SALES,
  6      MAX(Total_Amount) AS MAXIMUM_SALES
  7  FROM Orders;

TOTAL_ORDERS TOTAL_SALES AVERAGE_SALES MINIMUM_SALES MAXIMUM_SALES
------------ ----------- ------------- ------------- -------------
           6       12500       2083.33          1500          3000
```

2.1Customer-wise Sales Report

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name,
  4      c.Last_Name,
  5      COUNT(o.Order_ID) AS TOTAL_ORDERS,
  6      SUM(o.Total_Amount) AS TOTAL_PURCHASE
  7  FROM Customer c
  8  JOIN Orders o
  9  ON c.Customer_ID = o.Customer_ID
 10  GROUP BY
 11      c.Customer_ID,
 12      c.First_Name,
 13      c.Last_Name
 14  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID FIRST_NAME                     LAST_NAME
----------- ------------------------------ ------------------------------
TOTAL_ORDERS TOTAL_PURCHASE
------------ --------------
           1 Arun                           Kumar
            2           3200

           3 Karthik                        Raja
            1           3000

           5 Vijay                          Raj
            1           2500


CUSTOMER_ID FIRST_NAME                     LAST_NAME
----------- ------------------------------ ------------------------------
TOTAL_ORDERS TOTAL_PURCHASE
------------ --------------
           2 Priya                          Devi
            1           2000
```

3.Find Top customers based on purchase amount

```
SQL> SELECT *
  2  FROM
  3  (
  4      SELECT
  5          c.Customer_ID,
  6          c.First_Name,
  7          c.Last_Name,
  8          SUM(o.Total_Amount) AS TOTAL_PURCHASE
  9      FROM Customer c
 10      JOIN Orders o
 11      ON c.Customer_ID = o.Customer_ID
 12      GROUP BY
 13          c.Customer_ID,
 14          c.First_Name,
 15          c.Last_Name
 16      ORDER BY TOTAL_PURCHASE DESC
 17  )
 18  WHERE ROWNUM <= 3;

CUSTOMER_ID FIRST_NAME                     LAST_NAME
----------- ------------------------------ ------------------------------
TOTAL_PURCHASE
--------------
          1 Arun                           Kumar
          3200

          3 Karthik                        Raja
          3000

          5 Vijay                          Raj
          2500
```

4.Identify best-selling products

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(oi.Quantity) AS TOTAL_QUANTITY_SOLD
  5  FROM Product p
  6  JOIN Order_Item oi
  7  ON p.Product_ID = oi.Product_ID
  8  GROUP BY
  9      p.Product_ID,
 10      p.Product_Name
 11  ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT_ID PRODUCT_NAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       204 Shoes
                 3

       206 Kurti
                 2

       201 T-Shirt
                 2


PRODUCT_ID PRODUCT_NAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       205 Handbag
                 2

       202 Jeans
                 1
```

4.1Product sales amount

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(oi.Quantity) AS TOTAL_QUANTITY,
  5      SUM(oi.Subtotal) AS TOTAL_SALES
  6  FROM Product p
  7  JOIN Order_Item oi
  8  ON p.Product_ID = oi.Product_ID
  9  GROUP BY
 10      p.Product_ID,
 11      p.Product_Name
 12  ORDER BY TOTAL_SALES DESC;

PRODUCT_ID PRODUCT_NAME                                       TOTAL_QUANTITY
---------- -------------------------------------------------- --------------
TOTAL_SALES
-----------
       204 Shoes                                                           3
       4500

       203 Saree                                                           1
       2000

       205 Handbag                                                         2
       2000


PRODUCT_ID PRODUCT_NAME                                       TOTAL_QUANTITY
---------- -------------------------------------------------- --------------
TOTAL_SALES
-----------
       206 Kurti                                                           2
       1600

       201 T-Shirt                                                         2
```

5.Category-wise sales analysis

```
SQL> SELECT
  2      c.Category_ID,
  3      c.Category_Name,
  4      SUM(oi.Quantity) AS TOTAL_QUANTITY_SOLD,
  5      SUM(oi.Subtotal) AS TOTAL_SALES
  6  FROM Category c
  7  JOIN Product p
  8  ON c.Category_ID = p.Category_ID
  9  JOIN Order_Item oi
 10  ON p.Product_ID = oi.Product_ID
 11  GROUP BY
 12      c.Category_ID,
 13      c.Category_Name
 14  ORDER BY TOTAL_SALES DESC;

CATEGORY_ID CATEGORY_NAME                  TOTAL_QUANTITY_SOLD TOTAL_SALES
----------- ------------------------------ ------------------- -----------
        103 Footwear                                         3        4500
        102 Womens Wear                                      3        3600
        101 Mens Wear                                        3        2400
        104 Accessories                                      2        2000
```

6.Combined business sales report

```
SQL> SELECT
  2      (SELECT COUNT(*)
  3       FROM Orders) AS TOTAL_ORDERS,
  4    (SELECT NVL(SUM(Total_Amount), 0)
  5       FROM Orders) AS TOTAL_SALES,
  6    (SELECT NVL(ROUND(AVG(Total_Amount), 2), 0)
  7       FROM Orders) AS AVERAGE_SALES,
  8    (SELECT COUNT(*)
  9       FROM Customer) AS TOTAL_CUSTOMERS,
 10    (SELECT COUNT(*)
 11       FROM Product) AS TOTAL_PRODUCTS
 12  FROM DUAL;

TOTAL_ORDERS TOTAL_SALES AVERAGE_SALES TOTAL_CUSTOMERS TOTAL_PRODUCTS
------------ ----------- ------------- ---------------- --------------
           6       12500       2083.33                5              6
```